Experiment # 12

Introduction to the Parallel Computing using Message Passing Interface (MPI)

Objective:

Objective of this lab is to get the knowledge about Message Passing Interface MPI on multiple CPUs to distribute one task on multiple machines.

Software Tools:

- Ubuntu

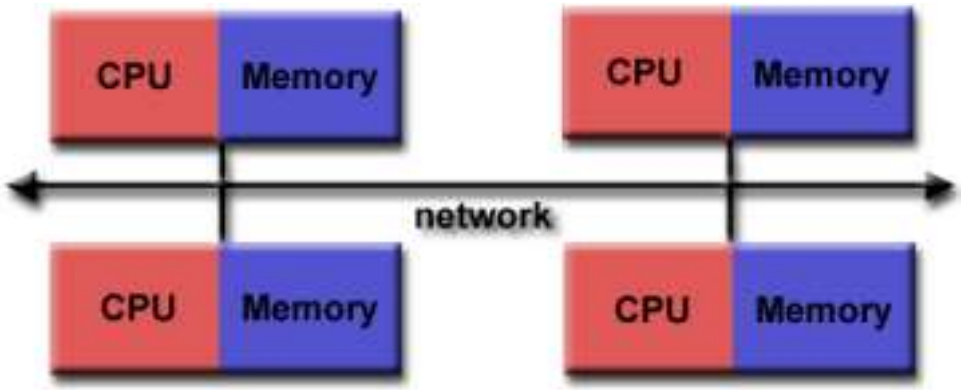
Theory:

MPI (Message Passing Interface)

MPI is a standard for implementing communication between computer nodes/processes in parallel programming. The goal of the Message Passing Interface is to establish a portable, efficient, and flexible standard for message passing that will be widely used for writing message passing programs. The goal of this lab is to familiarize with MPI how to develop and run parallel programs according to the MPI standard.

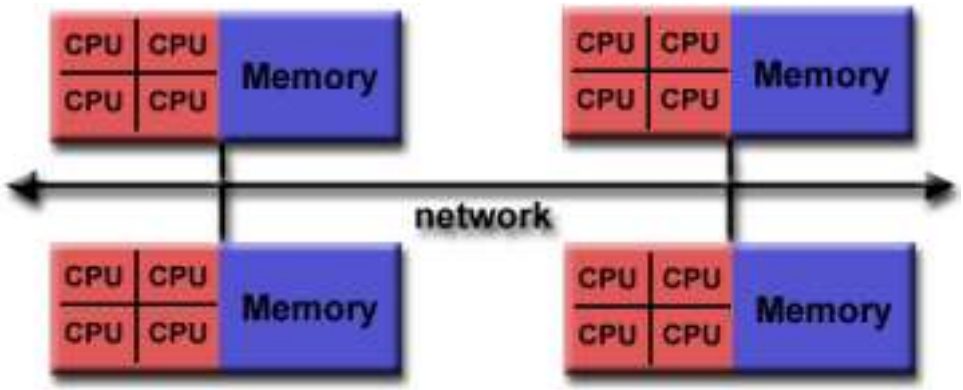
Programming Model:

Originally, MPI was designed for distributed memory architectures, which were becoming increasingly popular at that time (1980s - early 1990s).



As architecture trends changed, shared memory SMPs were combined over networks creating hybrid distributed memory / shared memory systems.

MPI implementers adapted their libraries to handle both types of underlying memory architectures seamlessly. They also adapted/developed ways of handling different interconnect and protocols.



Today, MPI runs on virtually any hardware platform:

- o Distributed Memory
- o Shared Memory
- o Hybrid

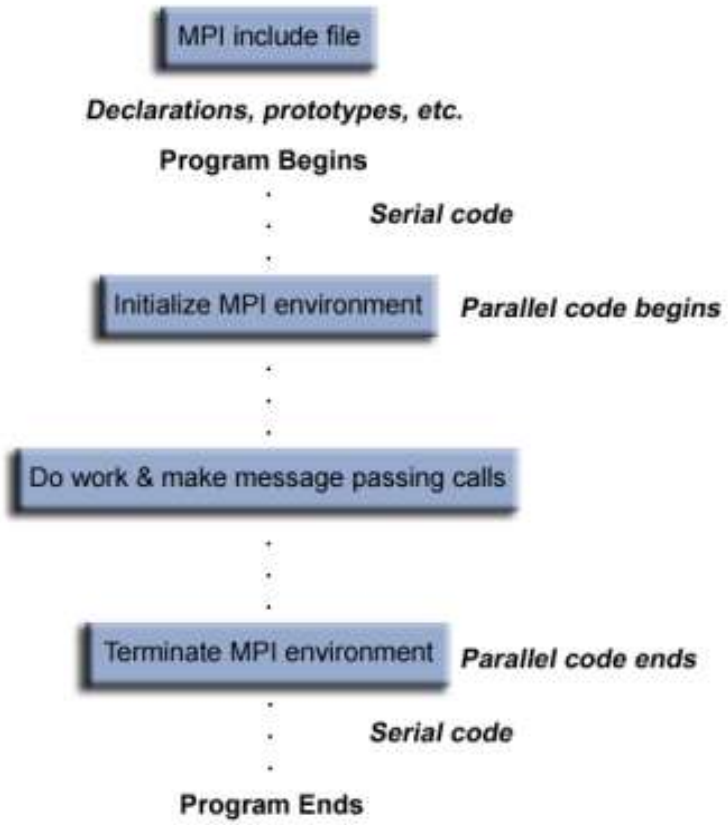
The programming model clearly remains a distributed memory model however, regardless of the underlying physical architecture of the machine.

All parallelism is explicit: the programmer is responsible for correctly identifying parallelism and implementing parallel algorithms using MPI constructs.

Reasons for Using MPI:

- **Standardization** - MPI is the only message passing library that can be considered a standard. It is supported on virtually all HPC platforms. Practically, it has replaced all previous message passing libraries.
- **Portability** - There is little or no need to modify your source code when you port your application to a different platform that supports (and is compliant with) the MPI standard.
- **Performance Opportunities** - Vendor implementations should be able to exploit native hardware features to optimize performance. Any implementation is free to develop optimized algorithms.
- **Functionality** - There are ***over 430*** routines defined in MPI-3, which includes the majority of those in MPI-2 and MPI-1.
- **Availability** - A variety of implementations are available, both vendor and public domain.

General MPI Program Structure:



Environment Management Routines

This group of routines is used for interrogating and setting the MPI execution environment, and covers an assortment of purposes, such as initializing and terminating the MPI environment, querying a rank's identity, querying the MPI library's version, etc. Most of the commonly used ones are described below.

MPI_Init

Initializes the MPI execution environment. This function must be called in every MPI program, must be called before any other MPI functions and must be called only once in an MPI program. For C programs, MPI_Init may be used to pass the command line arguments to all processes.

```
MPI_Init (&argc,&argv) MPI_INIT (ierr)
```

MPI_Comm_size

Returns the total number of MPI processes in the specified communicator, such as MPI_COMM_WORLD. If the communicator is MPI_COMM_WORLD, then it represents the number of MPI tasks available to your application.

```
MPI_Comm_size (comm,&size)
MPI_COMM_SIZE (comm,size,ierr)
```

MPI_Comm_rank

Returns the rank of the calling MPI process within the specified communicator. Initially, each process will be assigned a unique integer rank between 0 and number of tasks - 1 within the communicator MPI_COMM_WORLD. This rank is often referred to as a task ID. If a process becomes associated with other communicators, it will have a unique rank within each of these as well.

```
MPI_Comm_rank (comm,&rank)
MPI_COMM_RANK (comm,rank,ierr)
```

MPI_Abort

Terminates all MPI processes associated with the communicator. In most MPI implementations it terminates ALL processes regardless of the communicator specified.

```
MPI_Abort (comm,errorcode)
MPI_ABORT (comm,errorcode,ierr)
```

MPI_Get_processor_name

Returns the processor name. Also returns the length of the name. The buffer for "name" must be at least MPI_MAX_PROCESSOR_NAME characters in size. What is returned into "name" is implementation dependent - may not be the same as the output of the "hostname" or "host" shell commands.

```
MPI_Get_processor_name (&name,&resultlength)
MPI_GET_PROCESSOR_NAME (name,resultlength,ierr)
```

MPI_Get_version

Returns the version and subversion of the MPI standard that's implemented by the library.

```
MPI_Get_version (&version,&subversion)
MPI_GET_VERSION (version,subversion,ierr)
```

MPI_Initialized

Indicates whether MPI_Init has been called - returns flag as either logical true (1) or false (0). MPI requires that MPI_Init be called once and only once by each process. This may pose a problem for modules that want to use MPI and are prepared to call MPI_Init if necessary. MPI_Initialized solves this problem.

MPI_Initialized (&flag) **MPI_INITIALIZED (flag,ierr)**

MPI_Wtime

Returns an elapsed wall clock time in seconds (double precision) on the calling processor.

MPI_Wtime () MPI_WTIME ()

MPI_Wtick

Returns the resolution in seconds (double precision) of MPI_Wtime.

MPI_Wtick () MPI_WTICK ()

MPI_Finalize

Terminates the MPI execution environment. This function should be the last MPI routine called in every MPI program - no other MPI routines may be called after it.

MPI_Finalize () MPI_FINALIZE (ierr)

Example: Hello.c

```
#include <stdio.h>
#include <math.h>
#include <mpi.h>

int main(int argc, char *argv[ ])
{
int myid, numprocs;

MPI_Init (&argc, &argv) ;
```

```
MPI_Comm_size(MPI_COMM_WORLD, &numprocs) ;
MPI_Comm_rank(MPI_COMM_WORLD, &myid) ;
printf("Hello from %d\n", myid) ;
printf("Numprocs is %d\n", numprocs) ;

MPI_Finalize():
}
```

Examples: Environment Management Routines

```
// required MPI include file
#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[]) {
int numtasks, rank, len, rc;
char hostname[MPI_MAX_PROCESSOR_NAME];

// initialize MPI
MPI_Init(&argc,&argv);

// get number of tasks
MPI_Comm_size(MPI_COMM_WORLD,&numtasks);

// get my rank
MPI_Comm_rank(MPI_COMM_WORLD,&rank);

// this one is obvious
MPI_Get_processor_name(hostname, &len);
printf ("Number of tasks= %d My rank= %d Running on %s\n",
numtasks,rank,hostname);

// do some work with message passing

// done with MPI
MPI_Finalize();
}
```

To execute the mpi based code we have "mpicc" compiler in linux.

mpicc -o EXEC SourceFile
mpiexec ./EXEC

Lab Tasks:

1. Run and execute the above example code.